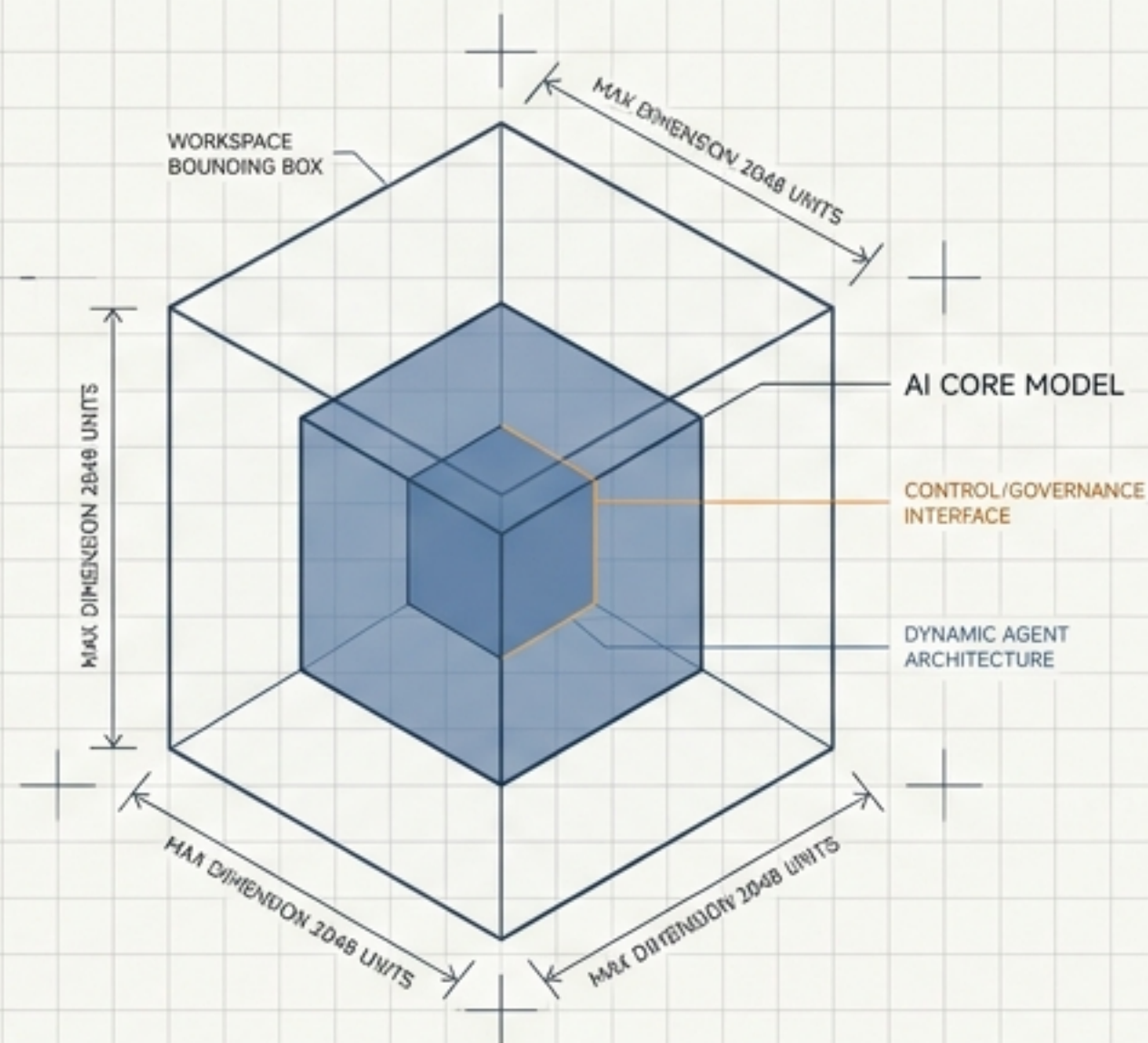


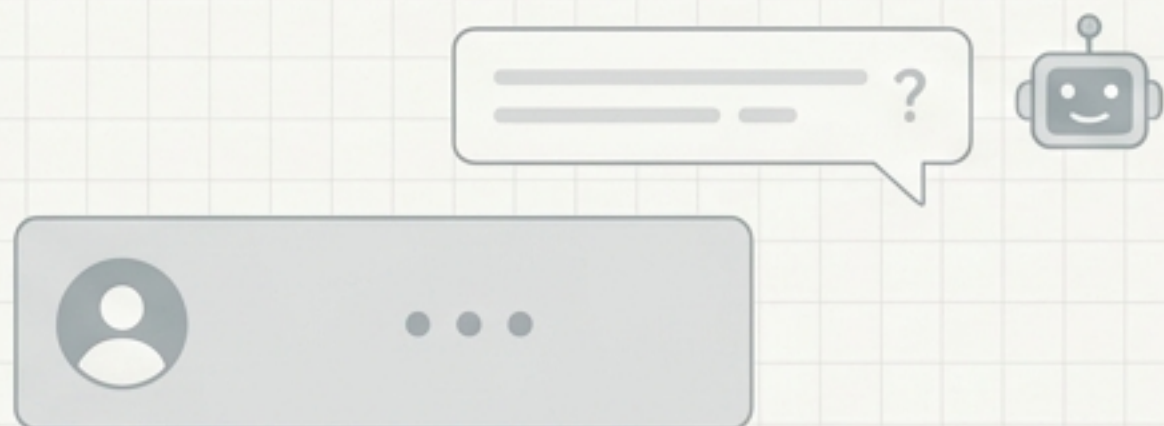


从 Agent 技术演进到 OpenCode 工程实践

构建现代 AI 工作系统的架构蓝图

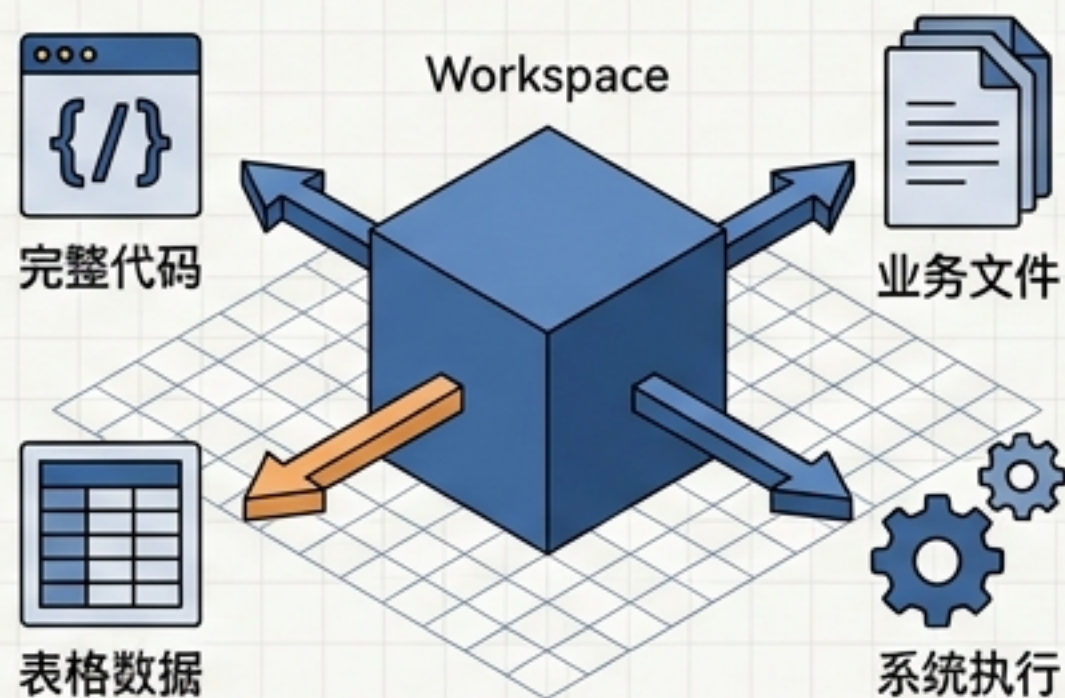
AI 应用正在从“信息系统”演进为“工作系统”

过去：对话与建议



- 交付一段文本回答
- 提供逻辑推理建议
- 代码片段需要人类复制执行

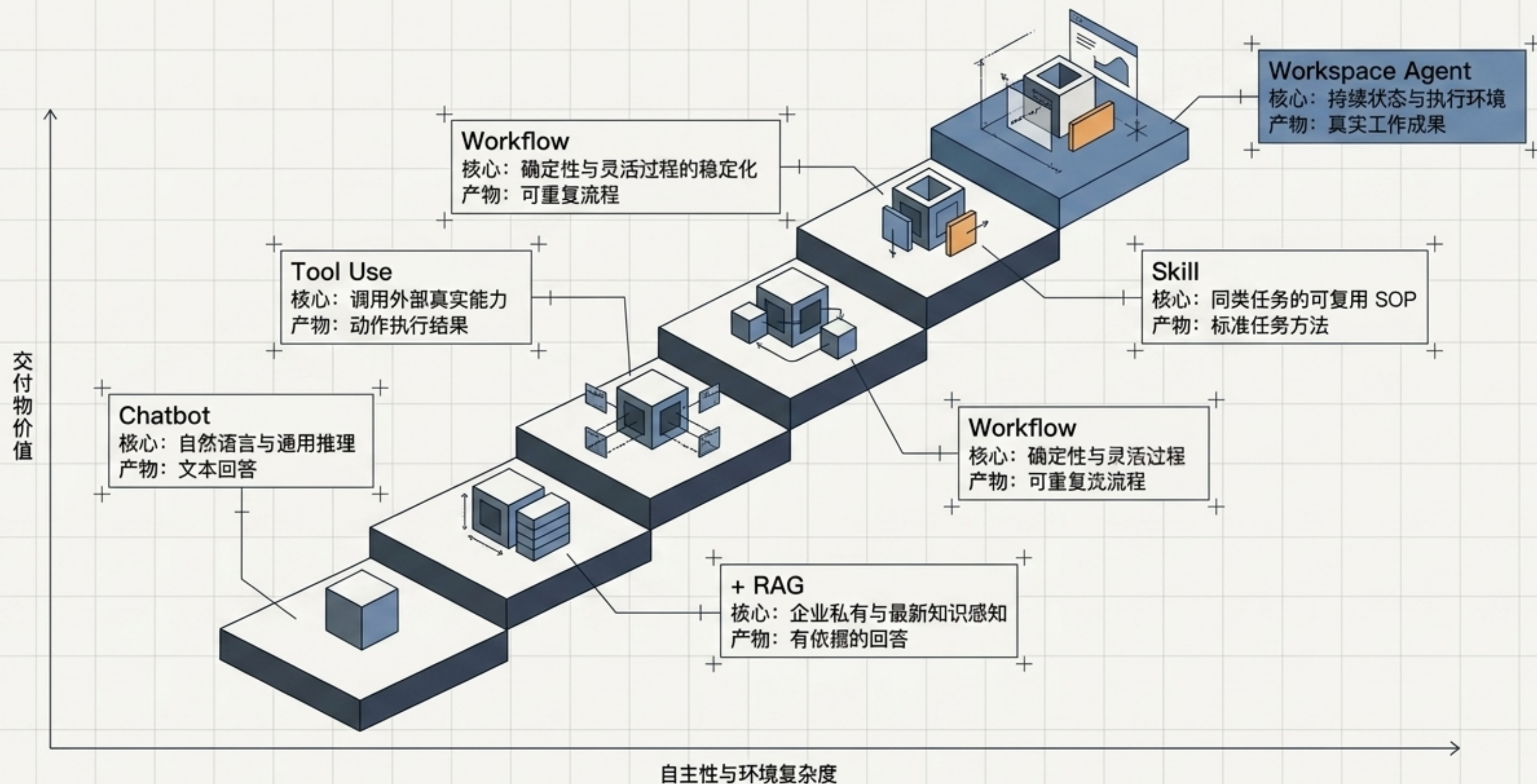
现在：真实工作成果



- 交付修改后的完整代码
- 处理完成的业务文件
- 可独立运行的工程项目
- 完成实际业务状态变更

评价标准已发生改变——从“回答是否自然流畅”转向“任务是否真正完成并经过验证”。

架构演进：从回答问题到完成任务



解析执行层：动作、流程与方法的本质区别

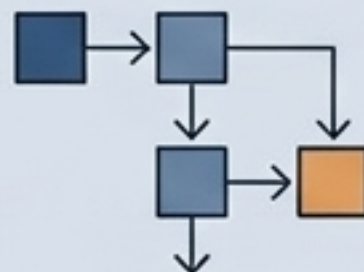


Tool / 工具

一句话定义：表达模型“可以做什么”。

本质：最小执行动作。

示例：读取文件、查询数据库、调用 API。

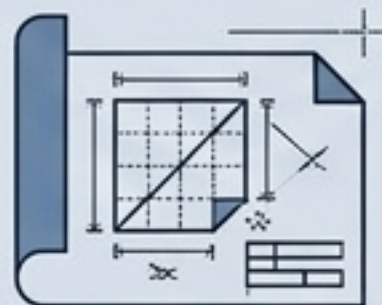


Workflow / 工作流

一句话定义：表达动作“按照什么确定顺序执行”。

本质：动作的固定排列与状态追踪。

示例：财务审批流、固定数据同步报表。



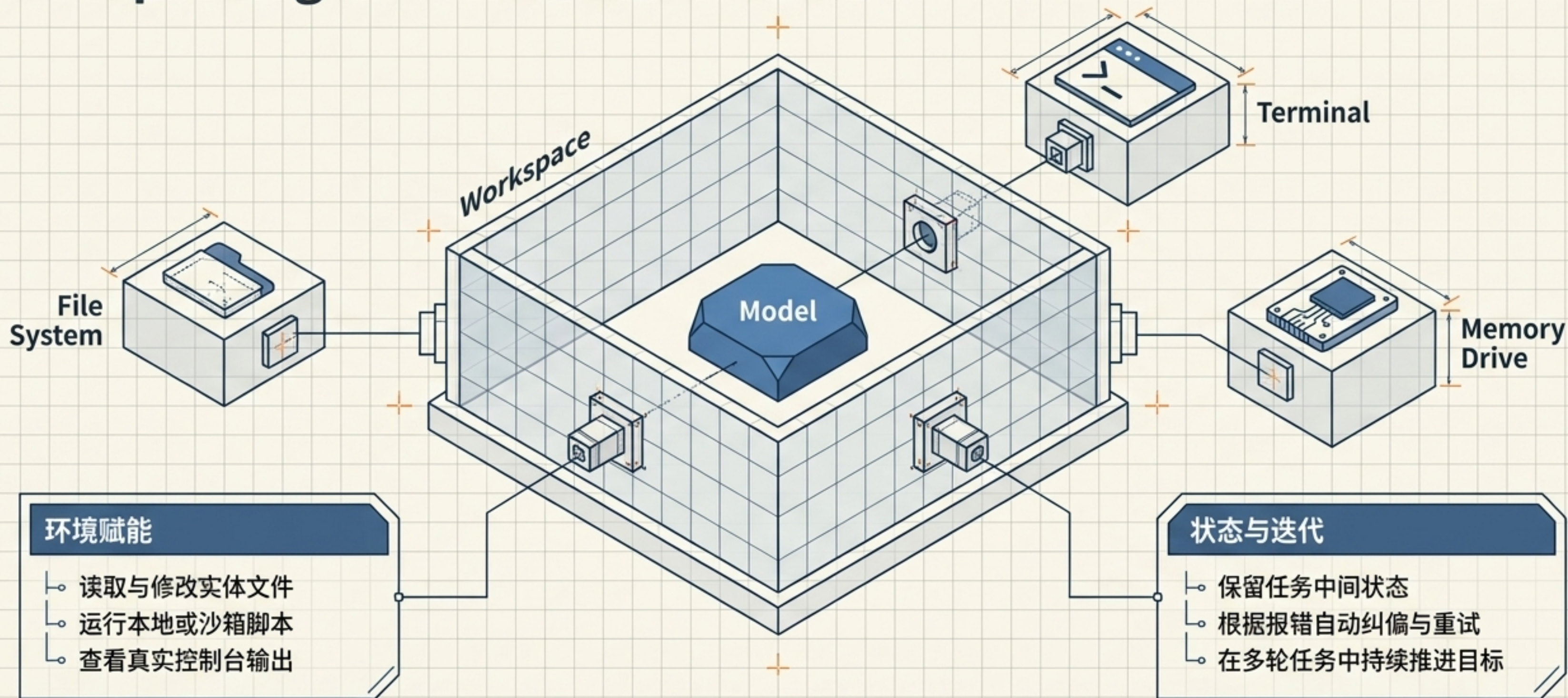
Skill / 技能

一句话定义：面对某一类任务时的“可复用做事方法”。

本质：带有适用条件、验证方法和异常处理的任务 SOP。

示例：代码 Review 规范、特定 Bug 排查指南。

Workspace Agent: 环境成为系统核心



核心洞察：AI 获得的不再只是知识和单个工具接口，而是一个“持续存在的工作空间”。对话轮数不再是关键，把事情做完才是目的。

系统的真实瓶颈正在从模型转向任务环境

模型推理 / 过去关注点

Prompt 怎么写？上下文窗口有多长？模型自身推理能力如何？



任务环境 / 当前核心故障点

找不到正确的文件路径或工作目录。

执行环境缺少必要的代码依赖。

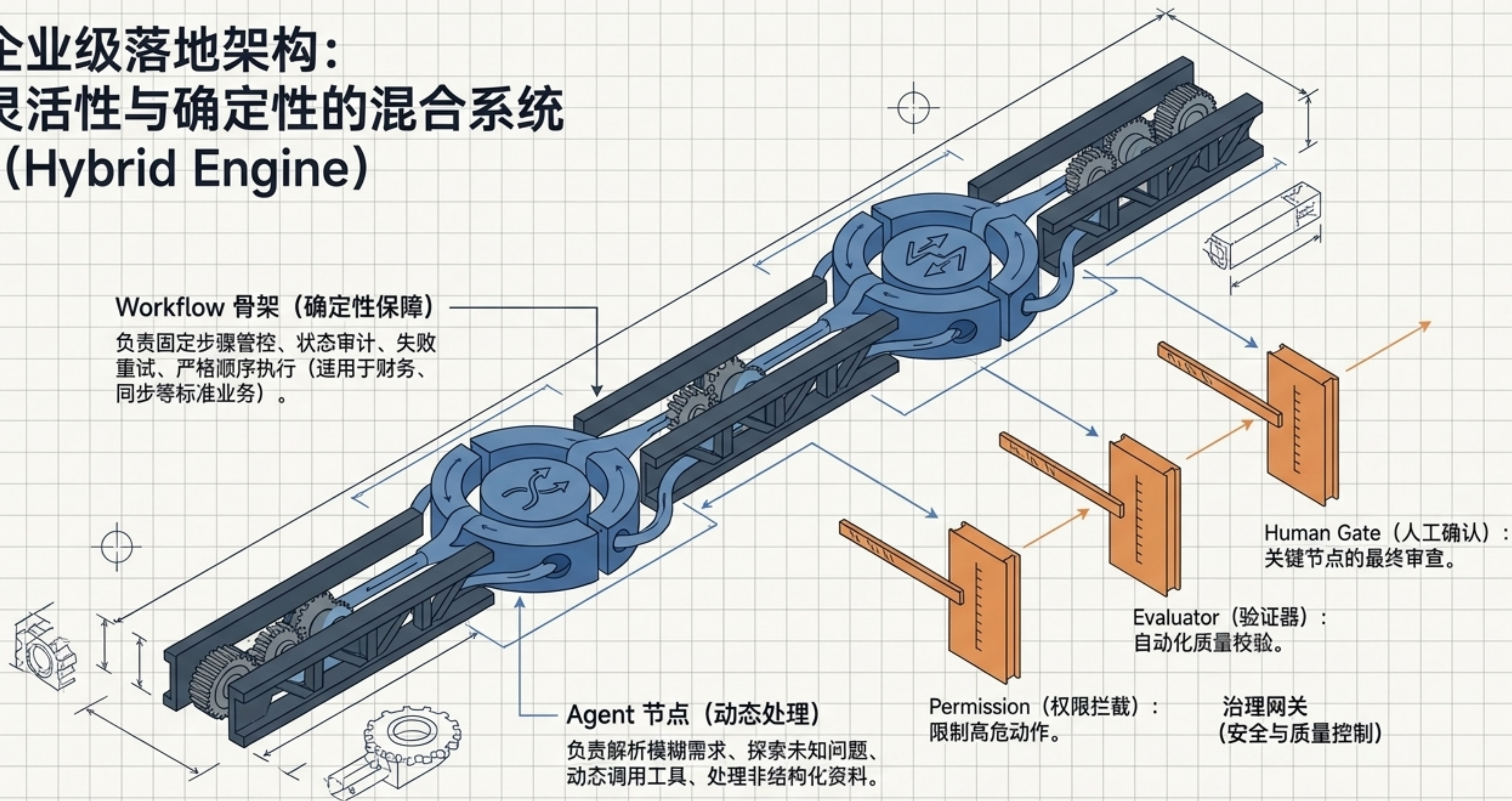
工具接口参数定义模糊不清。

上下文被历史无效操作污染、状态丢失。

执行后没有验证标准，无法安全回滚。

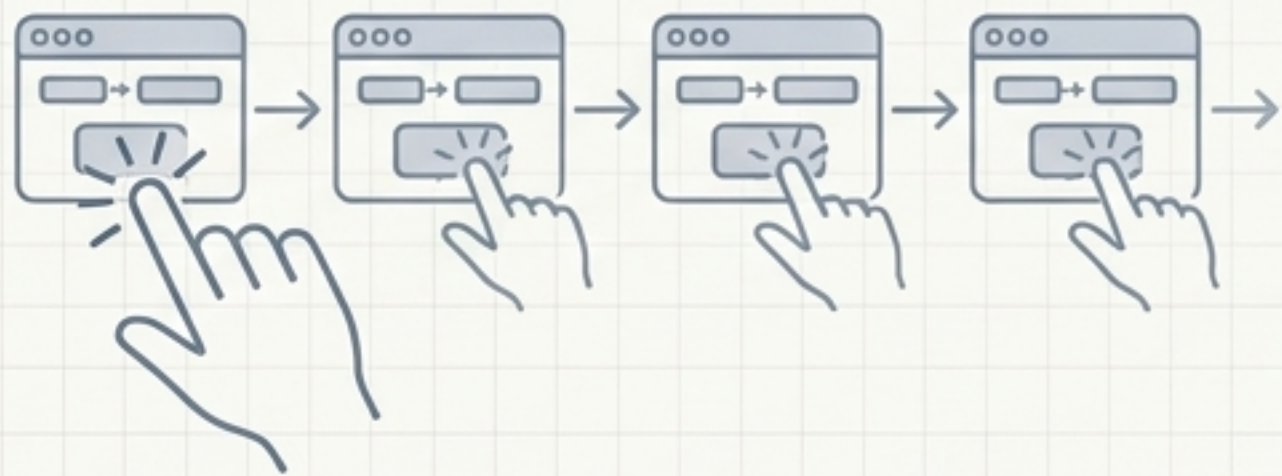
核心论断：很多任务失败并不是因为模型不会推理，而是因为外部环境没有准备好。未来应用间的差距将是工程系统的差距。

企业级落地架构： 灵活性与确定性的混合系统 (Hybrid Engine)



协作范式转移：人类角色的重新定义

旧模式（操作执行者）



传统软件与早期 Workflow 中，人类需要预先定义并手动操作每一个具体步骤。

新模式（任务导演与质量治理者）

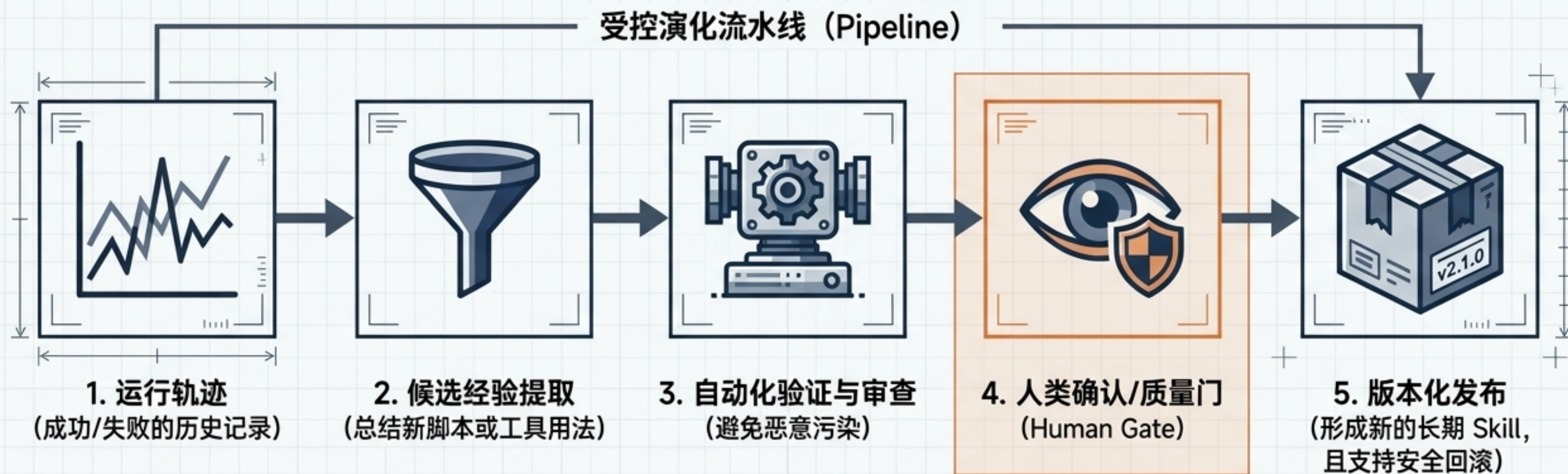


- ✓ 定义目标与交代背景（Context）。
- ✓ 设置权限边界与说明验收标准。
- ✓ 观察执行过程，随时暂停与接管。
- ✓ 提供最终体验与反馈。

“系统成熟不代表需要更少控制，而是控制方式从‘操作控制’升级为‘治理控制’。”

应用演化：从发布驱动走向受控的反馈驱动

问题背景：未经控制的自动学习会导致临时要求永久化、错误经验扩散、规则冲突与能力退化。

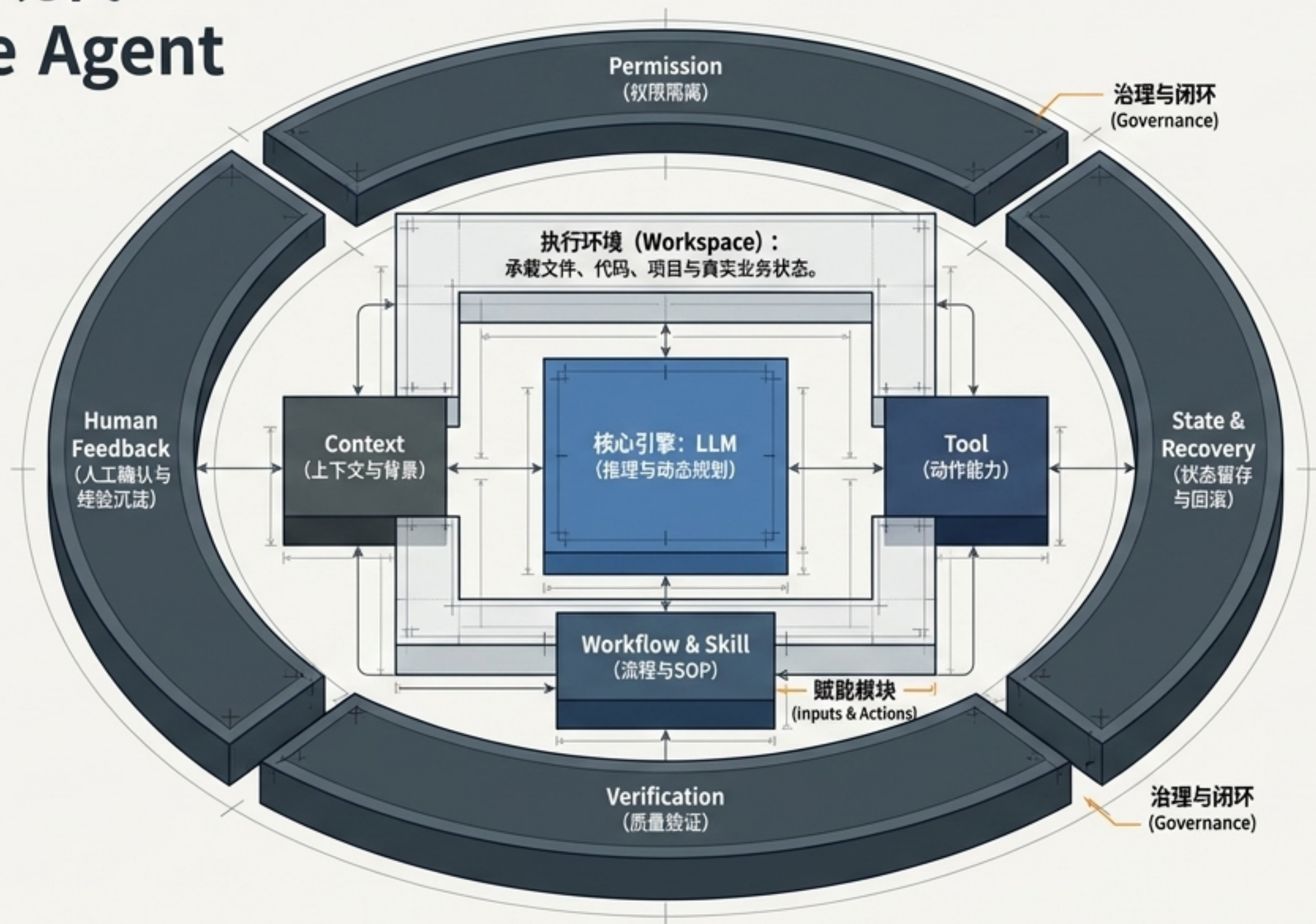


结论：真正需要的不是无约束的自动学习，而是可治理的能力迭代。

破除迷思：重新审视 Agent 系统的核心壁垒

旧有共识	工程真相
RAG 是核心壁垒。	向量库正成为基础能力。真正的价值在于知识是否可信、权限是否准确、检索时机与后续验证机制。
Prompt 是核心资产。	单独的 Prompt 无法构成系统。Workspace、工具契约、权限配置和恢复能力变得同等重要。
长上下文模型可以替代记忆与 Skill。	长窗口只解决“能不能装下”，无法解决“应该装什么”、“哪些已过期”、“哪些是可信的”。
Multi-Agent 是为了模拟人类组织。	Multi-Agent 首先是一种上下文隔离、权限隔离和执行隔离的系统工程机制，其次才是角色模拟。

终极蓝图：现代 Workspace Agent 的系统拓扑



总结陈词：成熟的 Coding Agent 不是一个更会写代码的大模型，而是一个由模型驱动、由工程保障的完整协作工作系统。